

Week 1 AI Learning Plan

Theory + Practice in parallel — 1.5 hrs/day — 7 days — Gemini API (free)

Deep Learning

NLP layer

LLM layer

YouTube

Blog

Course

Goal by end of Week 1

A single Jupyter notebook combining API calls, system prompts, token counting, embeddings with cosine similarity, and a multi-turn chatbot — all Week 1 concepts connected in one place, using the free Gemini API.

OVERVIEW — CLICK A DAY TO PLAN YOUR SESSION

Day 1	Neural networks (DL basics) First Gemini API call	~1.5 hrs
Day 2	Backprop + training System prompts and personas	~1.5 hrs
Day 3	RNN — DL meets sequences Tokenizer explorer	~1.5 hrs
Day 4	Attention mechanism Generate embeddings	~1.5 hrs
Day 5	Transformer architecture Few-shot + chain-of-thought	~1.5 hrs
Day 6	BERT vs GPT Chatbot with conversation memory	~1.5 hrs
Day 7	LLMs — scale and fine-tuning Week 1 combined project	~2 hrs

Day 1

Neural networks (DL basics) | First Gemini API call

~1.5 hrs

ORG CONNECTION:

Every AI project at your org calls an LLM exactly like this — you just built the foundation

Deep Learning

Theory — What is a neural network

Neurons, layers (input to hidden to output), weights and biases. How data flows forward through layers — called a forward pass. Key insight: a neural network is just a function that maps input numbers to output numbers. Open TensorFlow Playground alongside the video and see weights update in real time.

FOCUS ON

Neurons, layers, weights, forward pass, activation functions | What happens at each layer as data flows through | How TensorFlow Playground shows weights updating live

DON'T WORRY ABOUT YET

Exact math notation and matrix formulas | Why specific activation functions are chosen

RESOURCES

☒

YouTube

3Blue1Brown — But what is a neural network? (ch.1, 19 min)

☒

YouTube

3Blue1Brown — Gradient descent, how neural networks learn (ch.2)

☒

YouTube

StatQuest — Neural networks clearly explained

☒

Blog

TensorFlow Playground — visual neural net trainer

READ MORE — TOPIC REFERENCES

GeeksForGeeks

Neural Networks — A Beginners Guide

Javatpoint

Artificial Neural Network

TutorialsPoint

AI — Neural Networks

Analytics Vidhya

Basics of Neural Network

— Take a 10 min break before starting practice —

LLM layer

Practice — First Gemini API call

WATCH BEFORE CODING

YouTube

NEW SDK ✓

[Patrick Loeber — Gemini API with Python: Getting Started \(12 min\)](#)

YouTube

OLD SDK ■

[Gemini API with Python — Build a Customizable AI Chatbot](#)

OFFICIAL DOCS — NEW SDK (these always have correct syntax)

NEW SDK

[Official Gemini API Quickstart — New SDK](#)

NEW SDK

[Google GenAI Cookbook — all examples use new SDK](#)

COMMON ERRORS + QUICK FIXES

ModuleNotFoundError: google.genai → Run: `pip install google-genai --upgrade`**PermissionDenied 403 / Invalid API key** → Re-copy key from [aistudio.google.com](#) — no extra spaces**ResourceExhausted 429** → Free tier rate limit — wait 60 sec then retry**response.text is empty or None** → Prompt was blocked — rephrase it without sensitive words**Step 1** Setup

Install packages and get free API key.

```
pip install google-genai numpy jupyter notebook

# Get free API key: aistudio.google.com
# Sign in with Google -> Get API key -> Create -> Copy
```

Step 2 Basic call

Run this in your first code cell. You should see a response.

```
from google import genai

client = genai.Client(api_key="paste-your-key-here")

response = client.models.generate_content(
    model="gemini-2.0-flash",
    contents="What is a neural network? Explain in 3 sentences."
)
print(response.text)
```

Insight: Connection to theory: Gemini IS a neural network doing a forward pass on your text. You just triggered what you learned in theory.**Step 3** Check token usage

New cell. Understand what the API returns beyond just the text.

```
response = client.models.generate_content(
    model="gemini-2.0-flash",
    contents="What is deep learning in 2 sentences?"
)
print("Answer:", response.text)
print("Prompt tokens: ", response.usage_metadata.prompt_token_count)
print("Response tokens: ", response.usage_metadata.candidates_token_count)
print("Total tokens: ", response.usage_metadata.total_token_count)
```

Note: Token counts = how API costs are calculated in production. Knowing this prevents expensive surprises in Week 3 RAG work.

Step 4 Run 3 experiments

Each in a separate cell. Note how token count changes each time.

```
# Experiment 1: short question
r1 = client.models.generate_content(model="gemini-2.0-flash", contents="What is ML?")
print("Short Q tokens:", r1.usage_metadata.total_token_count)

# Experiment 2: ask for specific format
r2 = client.models.generate_content(
    model="gemini-2.0-flash",
    contents="Explain neural networks in exactly 5 bullet points"
)
print("Formatted Q tokens:", r2.usage_metadata.total_token_count)

# Experiment 3: your work domain
r3 = client.models.generate_content(
    model="gemini-2.0-flash",
    contents="How can AI be used in enterprise software projects? 3 examples."
)
print("Domain Q tokens:", r3.usage_metadata.total_token_count)
```

Step 5 Add system instruction — preview of Day 2

New cell. See how same question gives different output.

```
from google.genai import types

response = client.models.generate_content(
    model="gemini-2.0-flash",
    contents="What is a neural network?",
    config=types.GenerateContentConfig(
        system_instruction="You are a senior software engineer. Explain simply, short sentences."
    )
)
print(response.text)
```

Note: Same question, same model — completely different tone. This is prompt engineering. Day 2 goes deep into this.

End of Day 1 — mark each one when done

- ☒ I can explain what a neural network is in plain English without looking it up
- ☒ I understand what a forward pass is and know that the Gemini API runs one
- ☒ I saw weights update live in TensorFlow Playground with the spiral dataset
- ☒ My Gemini API key works and I ran a successful call
- ☒ I know what `prompt_token_count` and `total_token_count` mean
- ☒ I tested `system_instruction` and saw how it changes output style
- ☒ Notebook saved as `week1_day1.ipynb` with all code cells

TOMORROW: Day 2: How the network learns — backpropagation and gradient descent (theory). Build 3 different AI personas with system instructions and compare their outputs on the same question (practice).

MY NOTES — write your key takeaway before moving to the next day

Takeaway:

Surprised by:

Question:

Day 2Backprop + training | System prompts and personas~1.5 hrs

ORG CONNECTION: System instructions are how org chatbots are configured — one per use case

Deep Learning

Theory — How a neural network learns

Loss function measures how wrong the model is. Backpropagation traces which weights caused the error. Gradient descent nudges those weights to reduce error. Repeat millions of times. This is how every model — including every LLM — is trained. Today you understand the training loop that created Gemini.

FOCUS ON What the loss function measures and why it matters | How backprop flows error backwards through layers | What gradient descent actually does to the weights | Why training needs millions of iterations

DON'T WORRY ABOUT YET Calculus derivations of gradients | Specific optimisers like Adam or RMSProp

RESOURCES

☒

YouTube

3Blue1Brown — Backpropagation step by step (ch.3, 13 min)

☒

YouTube

3Blue1Brown — Backpropagation calculus (ch.4, 10 min)

☐

YouTube

StatQuest — Backpropagation clearly explained

☐

YouTube

Karpathy — micrograd: build backprop from scratch (optional, 2 hrs)

☒

Blog

LearnPrompting.org — system prompt basics

READ MORE — TOPIC REFERENCES

GeeksForGeeks

Backpropagation in Neural Network

Javatpoint

Backpropagation in Neural Network

GeeksForGeeks

Gradient Descent and its variants

TutorialsPoint

Gradient Descent

— Take a 10 min break before starting practice —

LLM layer

Practice — System prompts and 3 personas

WATCH BEFORE CODING

Free Course

OPENAI

DeepLearning.AI — Prompt Engineering for Developers (free, 1 hr)

YouTube

OLD SDK

Krish Naik — Prompt Engineering with Python

OFFICIAL DOCS — NEW SDK (these always have correct syntax)

NEW SDK

Gemini Prompting Strategies — Official Docs

NEW SDK

GenerateContentConfig reference — types module

COMMON ERRORS + QUICK FIXES**Model ignores system instruction** → Put `system_instruction` in `GenerativeModel()`, not `generate_content`**Output format not consistent** → Add more explicit examples and say "always follow this format exa**AttributeError on model variable** → You re-defined model — rename personas to `m1`, `m2`, `m3`**Step 1** Build 3 personas — same question, very different answers

Run each block. Compare the 3 outputs carefully.

```

from google import genai
from google.genai import types

client = genai.Client(api_key="your-key-here")
question = "What is machine learning?"

# Persona 1: Data Engineer
r1 = client.models.generate_content(
    model="gemini-2.0-flash", contents=question,
    config=types.GenerateContentConfig(system_instruction="You are a senior data engineer. Be technical, precise, concise")
print("DATA ENGINEER:\n", r1.text)

# Persona 2: Teacher
r2 = client.models.generate_content(
    model="gemini-2.0-flash", contents=question,
    config=types.GenerateContentConfig(system_instruction="You are a teacher for a 12-year-old. Use simple words only.")
print("\nTEACHER:\n", r2.text)

# Persona 3: Business Analyst
r3 = client.models.generate_content(
    model="gemini-2.0-flash", contents=question,
    config=types.GenerateContentConfig(system_instruction="You are a business analyst. Focus on ROI and enterprise use cases")
print("\nBUSINESS ANALYST:\n", r3.text)

```

Note: Observation: same model, same question — 3 completely different answers. The system instruction is the most powerful tool you have. This is exactly what org AI projects configure per use case.

Step 2 Control output format precisely

New cell. Tell the model exactly how to structure its answer.

```

from google.genai import types

fmt = "SUMMARY: [one sentence]\nKEY POINTS:\n- [point 1]\n- [point 2]\nUSE CASE: [one real-world example]"

for topic in ["neural networks", "gradient descent", "transformers"]:
    print(f"=== {topic.upper()} ===")
    r = client.models.generate_content(
        model="gemini-2.0-flash", contents=f"Explain {topic}",
        config=types.GenerateContentConfig(system_instruction=f"Always use this format:\n{fmt}")
    )
    print(r.text)
    print()

```

Insight: This format control is the foundation of every production LLM app. RAG systems, chatbots, and agents all rely on this to get predictable, parseable outputs.

Step 3 Few-shot prompting — teach by example

New cell. Give examples inside the prompt to control output.

```
from google import genai
from google.genai import types

client = genai.Client(api_key="your-key-here")

few_shot = """Classify the sentiment of each review.

Review: "The food was amazing and service was great!"
Sentiment: positive

Review: "The product broke after one day."
Sentiment: negative

Review: "Package arrived on time, nothing special."
Sentiment: neutral

Review: "Absolutely love this, best purchase ever!"
Sentiment: ""

result = client.models.generate_content(model="gemini-2.0-flash", contents=few_shot)
print(result.text)
```

Note: Notice the output mirrors the pattern exactly. Few-shot prompting works because you are literally showing the model examples of what you want. This is Day 5 prompt engineering used early.

End of Day 2 — mark each one when done

- ☒ I can explain loss, backprop and gradient descent in plain language
- ☒ I understand that LLMs were trained using this same process on trillions of tokens
- ☒ I built 3 different personas and compared their outputs on the same question
- ☒ I can control the exact output format of the model using system instructions
- ☒ I ran few-shot prompting and saw the model mirror my examples
- ☒ Notebook saved as week1_day2.ipynb

TOMORROW: Day 3: RNN — how DL was applied to text before transformers, and why it failed (theory). Tokenizer explorer — see how text becomes numbers and build a token counter function (practice).

MY NOTES — write your key takeaway before moving to the next day

Takeaway: _____

Surprised by: _____

Question: _____

Day 3 RNN — DL meets sequences | Tokenizer explorer

~1.5 hrs

ORG CONNECTION: Token counting before every LLM call prevents budget overruns in production org apps

Deep Learning

Theory — Why RNNs were built — and why they failed

Normal neural nets (Days 1-2) treat every input independently — they have no memory. RNNs added a hidden state that carries memory from one word to the next. But that memory fades over long sentences — the vanishing gradient problem. This is exactly the problem that attention (Day 4) was invented to fix. Understand the problem before you learn the solution.

FOCUS ON Why standard neural nets cannot handle sequences | How RNN hidden state carries memory forward | Why memory fades over long sentences (vanishing gradient) | What LSTM added to fix the memory problem

DON'T WORRY ABOUT YET LSTM gate equations and formulas | GRU architecture details

RESOURCES

☒ **YouTube** StatQuest — Recurrent neural networks clearly explained (15 min)

☒ **YouTube** StatQuest — LSTM step by step (25 min)

☐ **Blog** Karpathy — The unreasonable effectiveness of RNNs (karpathy.github.io)

☒ **Blog** OpenAI tokenizer — platform.openai.com/tokenizer (browser tool)

READ MORE — TOPIC REFERENCES

GeeksForGeeks

Introduction to Recurrent Neural Network

Javatpoint

Recurrent Neural Network

GeeksForGeeks

Long Short-Term Memory (LSTM)

Analytics Vidhya

Introduction to RNN

— Take a 10 min break before starting practice —

NLP layer **Practice — Tokenizer explorer — how text becomes numbers**

WATCH BEFORE CODING

YouTube

CONCEPTS

Karpathy — Lets build the GPT Tokenizer (watch first 30 min)

Browser Tool

CONCEPTS

OpenAI Tokenizer browser tool — no video needed

OFFICIAL DOCS — NEW SDK (these always have correct syntax)

NEW SDK

Token counting — Official Gemini Docs

NEW SDK

Rate limits and quotas — free tier details

COMMON ERRORS + QUICK FIXES

count_tokens returns 0 → Pass a string, not a list — client.models.count_tokens(model="gem

Tamil text shows 3x more tokens than English → Correct — non-English is expensive. Factor this into org app desi

Jupyter kernel crashes on large text → Text too big — truncate to first 1000 characters for testing

Step 1 Count tokens with Gemini — see how text is measured

New cell. Gemini has a built-in token counter — use it.

```

from google import genai

client = genai.Client(api_key="your-key-here")

def count_tokens(text):
    result = client.models.count_tokens(model="gemini-2.0-flash", contents=text)
    return result.total_tokens

tests = {
    "Simple English": "What is machine learning?",
    "Tamil text": "■■■■■■■■ ■■■■■■■ ■■■■■■■ ■■■■■■■?",
    "Python code": "def hello():\n    print('Hello World')",
    "Emojis": "Hello! How are you today?",
    "Repeated text": "machine learning " * 10,
}

for name, text in tests.items():
    t = count_tokens(text)
    print(f"{name:<20} {t} tokens")

```

Important: Tamil text uses significantly more tokens than English for the same meaning. Non-English content is more expensive and slower. This matters when building apps for Indian enterprises.

Step 2 Build a reusable token counter — use this for all future practice

This function saves money and prevents hitting token limits.

```

def smart_send(prompt, max_tokens=500):
    token_count = client.models.count_tokens(model="gemini-2.0-flash", contents=prompt).total_tokens
    print(f"Token check: {token_count} tokens")
    if token_count > max_tokens:
        print(f"WARNING: {token_count} exceeds limit {max_tokens}")
        return None
    response = client.models.generate_content(model="gemini-2.0-flash", contents=prompt)
    print(f"Tokens used: {response.usage_metadata.total_token_count}")
    return response.text

result = smart_send("Explain neural networks briefly")
if result:
    print("\nResponse:", result)

```

Insight: Use this function for the rest of Week 1. It builds the habit of checking token usage before every call — essential for production RAG systems.

Step 3 Explore the OpenAI tokenizer browser tool

Open platform.openai.com/tokenizer in your browser. Try these:


```
# Open browser: platform.openai.com/tokenizer
# Paste each of these and observe how they split:

# 1. Normal sentence
"The quick brown fox jumps over the lazy dog"

# 2. Code
"def calculate_similarity(vec1, vec2):"

# 3. Uncommon words
"Antidisestablishmentarianism is a long word"

# 4. Numbers
"The year 2024 has 365 days"

# Notice: spaces, punctuation, and word endings
# are often separate tokens.
```

Note: Tokens are not words. "running" might be 1 token but "antidisestablishmentarianism" is 5+ tokens. This is why LLM context limits are in tokens, not words.

End of Day 3 — mark each one when done

- ☒ I understand why RNNs were needed and what problem they solved
- ☒ I understand the vanishing gradient problem and why it made RNNs fail on long text
- ☒ I can count tokens before sending using `client.models.count_tokens()`
- ☒ I saw that Tamil text uses more tokens than English for the same content
- ☒ I built a `smart_send` function that checks tokens before every API call
- ☒ Notebook saved as `week1_day3.ipynb`

TOMORROW: Day 4: Attention mechanism — the most important concept in all of modern AI, how every word looks at every other word (theory). Generate embeddings with Gemini and compute cosine similarity — this is the math that powers RAG (practice).

MY NOTES — write your key takeaway before moving to the next day

Takeaway: _____

Surprised by: _____

Question: _____

Day 4 Attention mechanism | Generate embeddings

~1.5 hrs

ORG CONNECTION: Cosine similarity is the exact search algorithm your Week 3 RAG app will use on org documents

Deep Learning

Theory — The attention fix — the most important concept in modern AI

Attention solved RNNs forgetfulness by letting every word look at every other word directly — no chain of memory needed. Each word gets a score for how relevant every other word is to understanding it. Those scores are called attention weights. This is the single most important concept in all of modern AI. Every LLM you will ever use is built on this.

FOCUS ON Why attention removes the need for sequential processing | How Query, Key, Value work intuitively (not mathematically) | What attention weights mean — which words the model focuses on | Why this was a revolutionary improvement over RNNs

DON'T WORRY ABOUT YET The exact QKV matrix multiplication formulas | Scaled dot-product attention math

RESOURCES

- ☒ **YouTube** 3Blue1Brown — Attention in transformers, visually explained (27 min, MUST WATCH)
- ☒ **YouTube** Karpathy — Attention explained (15 min)
- ☐ **YouTube** StatQuest — Word embeddings clearly explained (15 min)
- ☒ **Blog** Jay Alammar — The Illustrated Word2Vec (jalammar.github.io)
- ☒ **Blog** OpenAI embeddings API docs — platform.openai.com/docs

READ MORE — TOPIC REFERENCES

GeeksForGeeks

Attention Mechanism in Neural Networks

Javatpoint

Attention Mechanism in Neural Network

GeeksForGeeks

Word Embeddings in NLP

Analytics Vidhya

Comprehensive Guide to Attention

— Take a 10 min break before starting practice —

NLP layer

Practice — Generate embeddings — the math behind RAG

WATCH BEFORE CODING

- YouTube** **OLD SDK** **Vector Search with Embeddings and Cosine Similarity**
- YouTube** **OLD SDK** **Embeddings and Cosine Similarity in LLM**

OFFICIAL DOCS — NEW SDK (these always have correct syntax)

NEW SDK

Gemini Embeddings — Official Docs

NEW SDK

gemini-embedding-001 model details

COMMON ERRORS + QUICK FIXES

numpy not found → Run: pip install numpy

embed_content returns dict not array → Wrap in np.array() — result["embedding"] is a list

Similarity score is negative (-0.3) → Valid — means very different concepts. Range is -1 to +1

invalid argument: request payload size → Text too long for embedding — trim to under 2000 chars

Step 1 Get your first embedding vector

New cell. See what a sentence looks like as numbers.

```
from google import genai
import numpy as np

client = genai.Client(api_key="your-key-here")

def get_embedding(text):
    result = client.models.embed_content(model="gemini-embedding-001", contents=text)
    return np.array(result.embeddings[0].values)

vec = get_embedding("cat")
print(f"Vector length: {len(vec)}")
print(f"First 5 values: {vec[:5].round(4)}")
print(f"This word is {len(vec)} numbers")
```

Note: Every word, sentence, or document in RAG is converted to a vector like this. Similarity search finds the closest vectors to your query — that is RAG.

Step 2 Compute cosine similarity — the core of RAG search

New cell. See why similar words are mathematically close.

```
def cosine_similarity(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

# Compare word pairs
word_pairs = [
    ("cat", "kitten"),
    ("cat", "dog"),
    ("cat", "car"),
    ("king", "queen"),
    ("python", "programming"),
    ("python", "snake"),
]

print(f"{\"Pair\":<25} {\"Similarity\":>10}\")
print("-" * 37)
for w1, w2 in word_pairs:
    e1 = get_embedding(w1)
    e2 = get_embedding(w2)
    sim = cosine_similarity(e1, e2)
    bar = "=" * int(sim * 20)
    print(f"{w1} vs {w2:<15} {sim:>8.3f} {bar}")
```

Insight: Cat-kitten will be very close (0.9+). Cat-car will be far (0.6-0.7). This is how RAG knows which document chunks are relevant to your question — it finds the closest vectors.

Step 3 Sentence-level similarity — closer to real RAG

New cell. Compare full sentences, not just words.

```
sentences = [
    "How do I reset my password?",
    "I forgot my login credentials",
    "What is the capital of France?",
    "Tell me about Paris landmarks",
]

query = "I cannot log into my account"
q_vec = get_embedding(query)

print(f"Query: {query}\n")
results = []
for s in sentences:
    sim = cosine_similarity(q_vec, get_embedding(s))
    results.append((sim, s))

results.sort(reverse=True)
for sim, sentence in results:
    print(f"{sim:.3f} {sentence}")
```

Note: The top result will be the login/password sentence. This is exactly what a RAG pipeline does when you ask a question — it finds the most similar document chunks using this same math.

End of Day 4 — mark each one when done

- ☒ I watched the 3Blue1Brown attention video — it is the most important video of the week
- ☒ I understand Query, Key, Value intuitively — even if not mathematically
- ☒ I got an embedding vector and know it represents text as numbers
- ☒ I computed cosine similarity and saw that cat-kitten is closer than cat-car
- ☒ I ran sentence-level similarity and saw how RAG search actually works
- ☒ I understand that embeddings + cosine similarity = the engine of Week 3 RAG work
- ☒ Notebook saved as week1_day4.ipynb

TOMORROW: Day 5: Transformer architecture — how attention layers stack into the model behind every LLM (theory). Few-shot prompting and chain-of-thought — the two techniques that cover 80% of real prompt engineering (practice).

MY NOTES — write your key takeaway before moving to the next day

Takeaway: _____
Surprised by: _____
Question: _____

Day 5

Transformer architecture | Few-shot + chain-of-thought

~1.5 hrs

ORG CONNECTION: Few-shot + CoT are the two techniques your org will use most in production prompt templates

Deep Learning Theory — The transformer — built from everything you now know

Transformer = attention (Day 4) + positional encoding + stacked layers. Positional encoding tells the model word order since attention alone has no sense of sequence. Stack 96 of these attention layers and you get GPT-4. Today Jay Alammar's Illustrated Transformer is your main reference — it's all diagrams, zero equations, and 30 minutes of reading.

FOCUS ON Why positional encoding is needed alongside attention | What encoder and decoder halves of the transformer do | How stacking many attention layers creates deeper understanding | Why this architecture replaced RNNs completely

DON'T WORRY ABOUT YET Layer normalisation formulas | Feed-forward network details within each block

RESOURCES

- ☒ **Blog** Jay Alammar — The Illustrated Transformer — jalammar.github.io (read this toda
- ☒ **YouTube** 3Blue1Brown — Transformers visually explained (26 min)
- ☒ **YouTube** 3Blue1Brown — How GPT works visually explained (27 min)
- ☐ **Course** DeepLearning.AI — ChatGPT Prompt Engineering for Developers (free, 1 hr)

READ MORE — TOPIC REFERENCES

GeeksForGeeks
Transformer Neural Network

Javatpoint
Transformer in Machine Learning

GeeksForGeeks
Prompt Engineering in AI

Analytics Vidhya
Prompt Engineering Tips and Techniques

— Take a 10 min break before starting practice —

LLM layer Practice — Few-shot prompting and chain-of-thought

WATCH BEFORE CODING

[Free Course](#)[OPENAI](#)[DeepLearning.AI — Prompt Engineering \(few-shot + CoT lessons\)](#)[YouTube](#)[OLD SDK](#)[Krish Naik — Prompt Engineering Tutorial \(few-shot + chain-of-thought\)](#)

OFFICIAL DOCS — NEW SDK (these always have correct syntax)

NEW SDK

[Gemini Prompting Guide — few-shot and CoT examples](#)

NEW SDK

[Structured output — get JSON from Gemini reliably](#)

COMMON ERRORS + QUICK FIXES

Model ignores few-shot pattern → Put examples closer to the question and add "follow this pattern"**Chain-of-thought gives wrong final answer** → Add "then state your final answer on a new line:" at the end**Output too long / too short** → Add "answer in under 3 sentences" or "be detailed" to control len**Step 1** Few-shot prompting — teach by example

Give the model examples to set the exact output pattern.

```
from google import genai
client = genai.Client(api_key="your-key-here")

few_shot_prompt = """Classify the priority of each support ticket.

Ticket: "System is completely down, no one can work"
Priority: CRITICAL

Ticket: "Dashboard loads slowly sometimes"
Priority: LOW

Ticket: "Cannot export reports to PDF"
Priority: MEDIUM

Ticket: "Login page throws error for 50% of users"
Priority: """

result = client.models.generate_content(model="gemini-2.0-flash", contents=few_shot_prompt)
print(result.text)
```

Insight: Output will be HIGH or CRITICAL. The model learned your classification pattern from 3 examples. This is the fastest way to get structured, consistent outputs without any training.**Step 2** Chain-of-thought — make the model think step by step

Add "think step by step" and compare answer quality.

```
# Without chain-of-thought
q = "A team has 8 developers. Each sprint is 2 weeks. They can complete 3 features per developer per sprint. How many fe

r1 = client.models.generate_content(model="gemini-2.0-flash", contents=q)
print("WITHOUT COT:")
print(r1.text)

r2 = client.models.generate_content(model="gemini-2.0-flash", contents=q + "\n\nThink step by step:")
print("\nWITH CHAIN-OF-THOUGHT:")
print(r2.text)
```

Note: The chain-of-thought version will be more accurate. By asking the model to show its reasoning, you force it to not skip steps. This is critical for complex business logic in enterprise AI apps.

Step 3 Combine both — few-shot chain-of-thought

The most powerful prompting technique for structured reasoning.

```
cot_few_shot = """
Analyse the risk of each project scenario.

Scenario: Team of 5, 3 month deadline, new technology stack
Thinking: Small team + tight deadline + unfamiliar tech = multiple risk factors compounding
Risk: HIGH

Scenario: Team of 10, 6 month deadline, well-known tech stack
Thinking: Adequate team size, comfortable timeline, familiar technology = well-managed project
Risk: LOW

Scenario: Team of 3, 2 month deadline, proven tech but complex integration
Thinking:
"""

result = client.models.generate_content(model="gemini-2.0-flash", contents=cot_few_shot)
print(result.text)
```

Insight: This combination — few-shot examples with chain-of-thought — is used in production AI systems for any task requiring structured reasoning. You just built what consulting teams use for AI-assisted analysis.

End of Day 5 — mark each one when done

- ☒ I read Jay Alammar's Illustrated Transformer — it is the clearest visual reference
- ☒ I understand why positional encoding is needed alongside attention
- ☒ I understand encoder vs decoder and what each half is good for
- ☒ I ran few-shot prompting and got the model to mirror my exact output pattern
- ☒ I compared answers with and without chain-of-thought and saw the quality difference
- ☒ I built a combined few-shot + chain-of-thought prompt
- ☒ Notebook saved as week1_day5.ipynb

TOMORROW: Day 6: BERT vs GPT — encoder vs decoder, why GPT generates token by token (theory). Build a chatbot that actually remembers the conversation — the foundation of LangChain memory (practice).

MY NOTES — write your key takeaway before moving to the next day

Takeaway:

Surprised by:

Question:

Day 6

BERT vs GPT | Chatbot with conversation memory

~1.5 hrs

ORG CONNECTION: Conversation memory is what your org chatbot needs — LangChain automates exactly this in Week 2

NLP layer

Theory — Encoder vs decoder — why GPT generates token by token

The transformer has two halves. BERT uses only the encoder — reads the whole sentence at once, good for understanding and classification. GPT uses only the decoder — reads left to right and predicts the next token, good for generation. LLMs like GPT-4, Gemini, and Claude are all decoder-only. This is why you see responses appear token by token.

FOCUS ON What encoder-only models are good for vs decoder-only | Why decoder models generate one token at a time | Why LLMs are stateless — they have no memory between calls | What a context window means and why it has a limit

DON'T WORRY ABOUT YET BERT pre-training details like masked language model | Token type embeddings

RESOURCES

☒

Blog

Jay Alammar — The Illustrated BERT (jalammar.github.io)

☒

YouTube

StatQuest — BERT clearly explained (20 min)

☒

YouTube

Karpathy — State of GPT (30 min — ties everything together)

☒

Blog

Google — Conversation history docs for Gemini API

READ MORE — TOPIC REFERENCES

GeeksForGeeks

Explanation of BERT Model in NLP

Javatpoint

BERT in NLP

GeeksForGeeks

OpenAI GPT Model

Analytics Vidhya

Demystifying BERT

— Take a 10 min break before starting practice —

LLM layer

Practice — Chatbot with conversation memory

WATCH BEFORE CODING

YouTube

NEW SDK ✓

Patrick Loeber — Gemini API (jump to 06:10 for chat section)

YouTube

OLD SDK ■

Build an AI Chatbot with Memory in Python

OFFICIAL DOCS — NEW SDK (these always have correct syntax)

NEW SDK

Gemini Chat / Multi-turn — Official Docs

NEW SDK

Context window and token limits

COMMON ERRORS + QUICK FIXES

Model forgets context after 5-6 turns → Context window limit — normal. Solution: summarise old history

chat.history grows huge → Expected — tokens accumulate. Print token count each turn to moni

KeyError on chat.history → Use chat.history[i].parts[0].text not direct dict access

Step 1 Build a chatbot that remembers — using Gemini chat session

New cell. This is the most important practice of the week.

```

from google import genai
from google.genai import types

client = genai.Client(api_key="your-key-here")

chat = client.chats.create(
    model="gemini-2.0-flash",
    config=types.GenerateContentConfig(
        system_instruction="You are a helpful AI learning assistant."
    )
)

questions = [
    "What is a transformer in AI?",
    "How does attention work inside it?",
    "What were the two things I just asked you about?",
]

for q in questions:
    print(f"You: {q}")
    response = chat.send_message(q)
    print(f"Gemini: {response.text[:250]}...")
    print(f"[History: {len(chat.get_history())} messages]\n")

```

Note: The third question tests memory. It will correctly say "transformer" and "attention". This works because Gemini sends the full chat history with every message — exactly what LangChain memory automates.

Step 2 See what history looks like under the hood

New cell. Understand what is actually being sent to the API.

```

# After running the chat above, inspect the history
print("Full conversation history sent to API each time:")
print(f"Total messages in history: {len(chat.history)}")
print()
for i, msg in enumerate(chat.history):
    role = msg.role.upper()
    text = msg.parts[0].text[:80]
    print(f"[{i+1}] {role}: {text}...")

```

Insight: Every message you see here is sent to the API on every call. As conversations get longer, token costs grow. LangChain provides smart ways to summarise or trim history. You'll use this in Week 2.

Step 3 Test the context limit — what happens when history gets too long

New cell. Build awareness of the context window.

```

chat2 = client.chats.create(model="gemini-2.0-flash")

for i in range(1, 8):
    q = f"Tell me one interesting fact about the number {i}"
    r = chat2.send_message(q)
    tokens = r.usage_metadata.total_token_count
    hist = len(chat2.get_history())
    print(f"Turn {i}: history={hist} msgs, total_tokens={tokens}")

```


Important: Token count grows with every turn. A long conversation can hit the context limit. This is why Week 2 LangChain covers memory management — summarising old history to keep costs low.

End of Day 6 — mark each one when done

- ☒ I understand encoder vs decoder and why Gemini/GPT/Claude are decoder-only
- ☒ I understand that LLMs are stateless and why history must be resent every time
- ☒ I built a chatbot that correctly remembers previous questions
- ☒ I inspected the history object and saw what is actually sent to the API
- ☒ I tracked how tokens grow with conversation length
- ☒ Notebook saved as week1_day6.ipynb

TOMORROW: Day 7: What makes an LLM — pretraining, fine-tuning, RLHF (theory). Week 1 combined project — one notebook with all concepts: system prompt + history + token counter + embeddings (practice). Show this to your team.

MY NOTES — write your key takeaway before moving to the next day

Takeaway: _____

Surprised by: _____

Question: _____

Day 7 LLMs — scale and fine-tuning | Week 1 combined project

~2 hrs

ORG CONNECTION: This combined notebook is your proof-of-concept — show it to your manager as Week 1 output

LLM layer Theory — What makes an LLM — the three stages

LLMs are large transformers trained in three stages. Pretraining: predict the next word across trillions of tokens from the internet — builds world knowledge. Fine-tuning: train on instruction-response pairs — makes it helpful and safe. RLHF: humans rate outputs and the model learns to score higher — aligns it with what people actually want. Watch Karpathy's State of GPT — it ties the entire week together in 30 minutes.

FOCUS ON What pretraining actually is — next token prediction at scale | Why fine-tuning changes behaviour without changing knowledge | What RLHF does and why it matters for helpfulness | Why bigger models are not always better for every task

DON'T WORRY ABOUT YET PPO and specific RL algorithms used in RLHF | Constitutional AI details

RESOURCES

- ☒ **YouTube** Karpathy — State of GPT (30 min — MUST WATCH today, ties whole week together)
- ☐ **YouTube** Karpathy — Intro to Large Language Models (1 hr overview)
- ☐ **YouTube** Karpathy — Build GPT from scratch (optional deep dive, 2 hrs)
- ☐ **Course** DeepLearning.AI short courses — bookmark for Week 2 onwards
- ☐ **Course** Hugging Face NLP course — free, start from Week 2

READ MORE — TOPIC REFERENCES

GeeksForGeeks

Large Language Model (LLM)

GeeksForGeeks

What is RLHF?

Analytics Vidhya

Introduction to Large Language Models

Towards DS

A Beginners Guide to LLMs

— Take a 10 min break before starting practice —

LLM layer

Practice — Week 1 combined project — all concepts in one notebook

WATCH BEFORE CODING

YouTube

CONCEPTS

Karpathy — State of GPT (30 min, ties whole week together)

YouTube

CONCEPTS

Balaji Chippada — 99% Learn AI Wrong (Telugu) — jump to 08:44

OFFICIAL DOCS — NEW SDK (these always have correct syntax)

NEW SDK

Full New SDK Reference — googleapis.github.io

NEW SDK

Google GenAI Cookbook — full examples repo

COMMON ERRORS + QUICK FIXES

Combined project runs but similarity is always 1.0 → You compared a vector to itself — use different texts for q_emb a

Token count grows each turn but embeddings slow down → Normal — embeddings call is separate from chat. Both are expected

One section works but another does not → Debug each section independently first, then combine

Step 1 The complete Week 1 project — show this to your team

One notebook connecting every concept from Days 1-6.

```

from google import genai
from google.genai import types
import numpy as np

client = genai.Client(api_key="your-key-here")

# ---- 1. SYSTEM INSTRUCTION CONFIG (Day 2) ----
sys_cfg = types.GenerateContentConfig(
    system_instruction="You are a helpful enterprise AI assistant. Be concise."
)

# ---- 2. TOKEN COUNTER (Day 3) ----
def count_tokens(text):
    return client.models.count_tokens(model="gemini-2.0-flash", contents=text).total_tokens

# ---- 3. EMBEDDINGS + SIMILARITY (Day 4) ----
def get_embedding(text):
    r = client.models.embed_content(model="gemini-embedding-001", contents=text)
    return np.array(r.embeddings[0].values)

def cosine_sim(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

# ---- 4. CHATBOT WITH MEMORY (Day 6) ----
chat = client.chats.create(model="gemini-2.0-flash", config=sys_cfg)

print("=" * 60)
print("WEEK 1 COMBINED PROJECT – New SDK")
print("=" * 60)

for q in ["What is RAG in AI systems?",
          "How does it use vector embeddings?",
          "What were the two topics I asked about?"]:
    tokens = count_tokens(q)
    print(f"\nYou ({tokens} tokens): {q}")
    response = chat.send_message(q)
    answer = response.text
    sim = cosine_sim(get_embedding(q), get_embedding(answer[:300]))
    print(f"Gemini: {answer[:200]}...")
    print(f"Tokens: {response.usage_metadata.total_token_count} | Sim: {sim:.3f}")
    print(f"History: {len(chat.get_history())} messages")

```

Insight: This notebook shows: system prompts controlling behaviour, token counting before every send, embeddings measuring question-answer relevance, and memory across turns. Every concept from Days 1-6, connected in one place.

End of Day 7 — mark each one when done

- ☒ I watched Karpathy's State of GPT — it ties the entire week together
- ☒ I can explain pretraining, fine-tuning, and RLHF in plain English
- ☒ I understand why RLHF makes models more helpful than raw pretraining
- ☒ My combined project notebook runs without errors
- ☒ The chatbot in the project correctly answers "what did I just ask about"
- ☒ Token counts and similarity scores print for every message
- ☒ I saved the project as week1_combined_project.ipynb
- ☒ I am ready to show this notebook to my team or manager

TOMORROW: Week 2: LangChain — chains, agents, and memory management. You will rebuild everything from Week 1 in a fraction of the code, and understand exactly why LangChain exists.

MY NOTES — write your key takeaway before moving to the next day

Takeaway: _____

Surprised by: _____

Question: _____

Bookmark these — use across all 4 weeks

The best free resources in the field. Revisit throughout your learning journey.

YouTube channels — best visual explanations (free)

- ☐ 3Blue1Brown — Neural nets, attention, transformers youtube.com/@3blue1brown
- ☐ Andrej Karpathy — Build LLMs and backprop from scratch youtube.com/@AndrejKarpathy
- ☐ StatQuest — Stats, ML, DL explained step by step youtube.com/c/joshstarmer
- ☐ Fireship — Quick visual explainers for tools like vector DBs youtube.com/@Fireship

Blogs — best written references (all free)

- ☐ Jay Alammar — Illustrated Transformer, BERT, GPT, Word2Vec jalammar.github.io
- ☐ Karpathy blog — Unreasonable effectiveness of RNNs karpathy.github.io
- ☐ Lilian Weng — Deep dives on attention, agents, diffusion lilianweng.github.io
- ☐ PromptingGuide.ai — Comprehensive prompt engineering reference promptingguide.ai
- ☐ LearnPrompting.org — Beginner-friendly prompt engineering guide learnprompting.org

Free courses — Week 1 onwards (no login or payment needed)

- ☐ DeepLearning.AI short courses — 1-2 hrs each, Andrew Ng deeplearning.ai/short-courses
- ☐ Hugging Face NLP course — transformers hands-on with code huggingface.co/learn/nlp-course
- ☐ fast.ai — Practical DL top-down, highly recommended course.fast.ai
- ☐ DeepLearning.AI — Prompt Engineering for Developers (Week 1) deeplearning.ai
- ☐ DeepLearning.AI — LangChain for LLM App Development (Week 2) deeplearning.ai
- ☐ DeepLearning.AI — LangChain: Chat with your data / RAG (Week 3) deeplearning.ai

Free tools — keep these open always

- ☐ OpenAI Tokenizer — paste text, see token splits live platform.openai.com/tokenizer
- ☐ TensorFlow Playground — train visual neural net in browser playground.tensorflow.org
- ☐ Google AI Studio — Gemini API key and free playground aistudio.google.com
- ☐ TF Embedding Projector — visualise word vectors in 3D projector.tensorflow.org
- ☐ ChromaDB — free local vector database for RAG (Week 3) docs.trychroma.com

Prompt engineering references — for Week 1 Day 2 and 5

- ☐ promptingguide.ai — DAIR.AI, the most complete free guide
- ☐ learnprompting.org — beginner friendly, structured topics
- ☐ anthropic.com/research/claude-prompting — Claude-specific tips